

第七章 链接

链接器的两个阶段：符号解析、重定位

符号：对应于一个函数、一个全局变量或一个静态变量（注意，这里是C 语言中任何以static 属性声明的变量）

强符号，弱符号在链接时候的三种规则，会判断
强符号：函数和初始化全局变量；弱符号：未初始化的全局变量
规则 1:不允许多个同名的强符号，每个强符号只能定义一次；否则， 链接器错误
规则 2:若有一个强符号和多个弱符号同名，则选择强符号；对弱符号的引用解析为强符号
规则 3:如果有多个弱符号，选择任意一个

简答：简述静态库及其优点。
答：编译时将所有相关的目标模块打包成为一个单独的文件用作链接器的输入，这个文件称为静态库。使用时，通过把相关函数编译为独立模块，然后封装成一个单独的静态库文件，应用程序可以通过在命令行上指定单独的文件名来使用这些在库中定义的函数，链接器只需复制被程序引用的目标模块，减少了可执行文件在磁盘和内存中的大小；同时，应用程序员只需包含较少的库文件的名字。

简答：简述链接器在用目标文件生成可执行文件时如何处理符号引用/解析的。
答：1) 全局符号中：有初值的全局变量、函数是强符号，无初值的全局变量是若符号
2) 根据强弱符号类别，确定被引用的符号在哪里
3) 根据各目标文件的段落大小、系统代码等信息，合并同类型的段，并计算可执行文件各段的大小，
4) 根据程序的内存映像，确定被引用的符号在程序运行时的内存地址
5) 根据在磁盘上的可执行文件中，各段落的大小，确定符号引用的位置
6) 根据上述信息，计算符号引用处应该采用的数值，并写入文件。

简答：共享库（动态链接库）的定义及实现方法。
答：共享库（动态链接库）是一个.so 的目标模块（elf 文件），在运行或加载时，由动态链接器
程序加载到任意的内存地址，并和一个和内存中的程序（如当前可执行目标文件）动态完全连接为一个可执行程序。使用它可节省内存与硬盘空间，方便软件的更新升级。如标准 C 库 libc.so 。
动态链接的实现方法
加载时动态链接：应用程序第一次加载和运行时，通过 ld-linux.so 动态链接器重定位动态库的代码和数据到某个内存段，再重定位当前应用程序中对共享库定义的符号的引用，然后将控制传递给应用程序（此后共享库位置固定了并不变）。
运行时动态链接：在程序执行过程中，通过 dlopen/dlsym 函数加载和连接共享库，实现符号重定位，通过 dlclose 卸载动态库。

重要题型：结合汇编和机器码计算重定位相对地址和绝对地址

库打桩：允许截获对共享库函数的调用，取而代之执行自己的代码。具体分为以下三种：
1) 编译时:源代码被编译时
2) 链接时:当可重定位目标文件被静态链接来形成一个可执行目标文件时
3) 加载/运行时:当一个可执行目标文件被加载到内存中，动态链接，然后执行时

.text节: 已编译程序的机器代码
.data节: 已初始化的全局和静态C变量
.bss: 未初始化的全局和静态C变量
有时候还这样进一步划分：
1) COMMON 未初始化的全局变量
2) .bss 未初始化的静态变量，以及初始化为0的全局或静态变量